

Documento de Padrão de Projeto

Projeto: Vireya

Versão do Documento: 1.2

Responsável pela Documentação: Vitor Carvalho

Introdução

Durante o desenvolvimento web, foi identificado que a lógica de acesso à API (CRUD) para diferentes entidades (como produtos, funcionários, etc.) estava sendo repetida em diversos services. Para eliminar essa duplicidade, foi criada uma abstração reutilizável baseada em padrões de projeto, facilitando a manutenção, escalabilidade e padronização do código.

Estrutura da Solução

A abstração foi implementada por meio de uma classe genérica chamada `CrudService<T>`, que define as operações padrão de CRUD utilizando o axios. Essa classe é estendida por serviços concretos, específicos de cada entidade, como `FuncionarioService`.

Classe Base: `CrudService.ts`

```
export abstract class CrudService<T extends { id: number }> {  
  protected abstract basePath: string;  
  
  async listar(): Promise<T[]> {  
    const { data } = await api.get<T[]>(`${this.basePath}/listar`);  
    return data;  
  }  
  
  async listarId(id: number): Promise<T> {  
    const { data } = await api.get<T>(`${this.basePath}/${id}`);  
    return data;  
  }  
  
  async criar(payload: Omit<T, "id">): Promise<T> {  
    const { data } = await api.post<T>(this.basePath, payload);  
    return data;  
  }  
  
  async atualizar(id: number, payload: Omit<T, "id">): Promise<T> {  
    const { data } = await api.put<T>(`${this.basePath}/atualizar/${id}`, payload);  
    return data;  
  }  
}
```

```

    async deletar(id: number): Promise<void> {
      await api.delete(`${this.basePath}/deletar/${id}`);
    }
  }
}

```

Subclasse Concreta: FuncionarioService.ts

```

import { CrudService } from "../CrudService";
import type { Funcionario } from "../types/Funcionario";
import api from "../axios/api";

class FuncionarioService extends CrudService<Funcionario> {
  protected basePath = "/funcionario";

  async getUserDataByEmail(email: string) {
    try {
      const response = await api.get(`/funcionario/email`, {
        headers: {
          "email": email,
        },
      });

      return response.data; // Retorna nome e cargo
    } catch {
      throw new Error("Erro na coleta de dados do usuário na requisição ao servidor");
    }
  }
}

export default new FuncionarioService();

```

Métodos adicionais, além do CRUD básico, podem ser adicionados aqui conforme necessário para operações específicas do funcionário (adicionei o `getUserDataByEmail` por exemplo).

Padrões de Projeto Aplicados

- **Template Method:** Define o esqueleto de uma operação em uma classe base, permitindo que subclasses personalizem partes específicas.

- **Repository Pattern:** Abstrai o acesso aos dados via HTTP e expõe uma interface simples de operações, promovendo separação entre lógica de negócio e infraestrutura.

Conclusão

A adoção dessa arquitetura trouxe mais organização e eficiência ao código. A separação de responsabilidades e o uso de abstrações reutilizáveis tornam o sistema mais coeso e menos acoplado.